Edge cases (where the ¾-word rule breaks) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — Why the rule breaks, and where
- Hint 2 — Worked examples, and the real numbers

Hint 1 — Why the rule breaks, and where

The tokenizer's vocabulary is built by a statistical process (byte-pair encoding, roughly: repeatedly merging the most frequent adjacent character pairs seen in training) run over a training corpus that's mostly English prose. Whatever pattern shows up often in that corpus earns a short, efficient token. Whatever doesn't — rare words, punctuation-heavy code syntax, non-Latin scripts — gets broken down into smaller, less efficient pieces, because the exact chunk wasn't common enough in training to deserve its own single token.

For code: count the punctuation characters separately from the words — parentheses, quotes, colons, and brackets each often become their own token, which is why code "looks short" but tokenizes long.

For non-English text: don't try to guess from an English rule of thumb at all here — just note that the same *meaning*, in a language with less training data behind it, usually costs noticeably more tokens than the English equivalent.

Write your guesses for the code snippet and the non-English sentence, in tokens, before checking them.

The deeper pattern connecting both exceptions (code, non-English text) is the same: **token efficiency tracks training-data frequency, not human-perceived complexity.** A short Urdu sentence isn't linguistically more complex than its English translation — it just has far less representation in the corpus the tokenizer's vocabulary was built from. Given this, predict: would a JSON-heavy system prompt (lots of `{`, `}`, `"`, `:`) tokenize better or worse than the equivalent information written as plain English sentences? Write your prediction, then check it against a real tokenizer.

Hint 2 — Worked examples, and the real numbers

```
code: "print('hello world')"
naive word-count estimate: ~3 tokens
actual (tokenizer tool): ~5-7 tokens
gap explained by: print, (, ', hello, world, ', ) each frequently tokenizing
                  separately or near-separately — punctuation isn't "free"

non-English: "میں آج بازار جا رہا ہوں"
English equivalent: "I am going to the market today" (~7 tokens)
actual for the Urdu sentence: often 15-20+ tokens
gap explained by: far less Urdu text in the tokenizer's training data means
                  the vocabulary has few or no efficient multi-character
                  chunks for this script — most of it falls back to small,
                  inefficient sub-word or byte-level pieces
```

Run your own two examples through the real tokenizer tool now and record the actual counts. Write down which parts of your own future projects fall into these exception categories: system prompts full of JSON schema examples, a multilingual support bot, or a feature that echoes back user-pasted code, then compare against the Solution.

Full solution: Show me the solution